PRÁCTICA: “2048”

Estructura de Computadores

Grado en Ingeniería Informática

feb26-jun26

Estudios de Informática, Multimedia y Telecomunicaciones

EIMT.UOC.EDU

Universitat Oberta de Catalunya

Presentación

La práctica que se describe a continuación consiste en la programación en lenguaje ensamblador x86_64 de un conjunto de subrutinas, que se tienen que poder llamar desde un programa en C

Competencias

Las competencias específicas que persigue la PRÁCTICA son:

- [13] Capacidad para identificar los elementos de la estructura y los principios de funcionamiento de un ordenador.
- [14] Capacidad para analizar la arquitectura y organización de los sistemas y aplicaciones informáticos en red.
- [15] Conocer las tecnologías de comunicaciones actuales y emergentes y saberlas aplicar convenientemente para diseñar y desarrollar soluciones basadas en sistemas y tecnologías de la información.

Objetivos

Introducir al estudiante en la programación de bajo nivel de un computador, utilizando el lenguaje ensamblador de la arquitectura Intel x86-64 y el lenguaje C.

Recursos

Podéis consultar los recursos del aula, pero no podéis hacer uso intensivo del foro.

El material básico que podéis consultar es:

- Módulo 6: Programación en ensamblador (x86_64)
- Documento “Entorno de trabajo”

Desarrollo

La práctica se divide en dos partes:

- Primera parte obligatoria:

  Implementar en lenguaje ensamblador las subrutinas correspondientes a las funcionalidades básicas de la práctica.
- Segunda parte opcional:

  Implementar en lenguaje ensamblador las subrutinas correspondientes a las funcionalidades adicionales necesarias para completar todas las funcionalidades de la práctica. Trabajar el paso de parámetros entre subrutinas modificando la implementación hecha en la primera parte.

Para cada una de las dos partes os proporcionaremos dos archivos: un archivo de código C y un archivo de código ensamblador.

El archivo de código C contiene una versión completa de la práctica para que os sirva de guía durante la implementación de las subrutinas en ensamblador, y también os permite ejecutar el juego para ver cómo funciona. Este archivo **no lo podéis modificar**.

Las variables globales utilizadas en la práctica están definidas en el código C. **No podéis añadir otras variables**.

Para acceder a los vectores y matrices en ensamblador tenéis que usar direccionamiento relativo o direccionamiento indexado: [vector+rsi], [rbx+rdi].

El archivo de código ensamblador contiene algunas subrutinas ya implementadas que **no podéis modificar** y otras que tenéis que implementar vosotros. En las cabeceras de cada subrutina encontraréis la información detallada para implementarlas. No se pueden definir otras subrutinas.

Para ayudaros en el desarrollo de la práctica disponéis de un menú con diferentes opciones para llamar a cada una de las subrutinas que tenéis que implementar, una opción correspondiente al juego completo llamando a las subrutinas en ensamblador que tenéis que implementar, y otra opción con el juego implementado en código C que os damos hecho. Se recomienda desarrollar la práctica siguiendo el orden de las opciones de este menú.

En el código C, donde se llama a las subrutinas de ensamblador que tenéis que implementar, encontraréis comentadas las llamadas a las funciones de C equivalentes. Si queréis probar les funcionalidades hechas en C lo podéis hacer quitando el comentario de la llamada de C y poniéndolo en la llamada a la subrutina de ensamblador.

Por ejemplo:

```
//=====================================================
subrutina();
//subrutina_C(); //=====================================================
```

El código hace una crida a la subrutina de ensamblador, podemos cambiar el comentario y llamar a la función de C.

```
//=====================================================
//subrutina();
subrutina_C(); //=====================================================
```

Recordad volver a dejar el código como estaba para probar vuestras subrutinas.

La Práctica: “2048”

La práctica consiste en implementar el juego del “2048” que consiste en ir moviendo los valores que se encuentran en un tablero de 4 x 4 de forma que cuando dos valores iguales entran en contacto se suman, hasta llegar a que en una casilla el valor sea igual a 2048. Se pueden desplazar los valores del tablero a la izquierda, derecha, arriba o abajo, al hacer un desplazamiento se arrastran todos los valores del tablero en la dirección seleccionada, eliminando los espacios en blanco que haya entre las casillas. Si al hacer el desplazamiento quedan dos casillas consecutivas con el mismo valor, se suman los valores, dejando una casilla con el valor sumado y la otra casilla en blanco.

Al finalizar un movimiento el programa genera un número 2 o un 4 que se añade al tablero en una casilla vacía seleccionada aleatoriamente.

Podéis ver una descripción más detallada del juego en:

https://ca.wikipedia.org/wiki/2048_(videojuego)

Y podéis ver una versión online del juego en la página de su autor:

http://gabrielecirulli.github.io/2048/

También hay una versiones del juego por iOS y Android.

En esta primera parte el juego del 2048 no estará completamente implementado. En la segunda parte opcional se implementarán las funcionalidades necesarias para tener un juego totalmente funcional.

Las subrutinas que hay que implementar en ensamblador para la Primera Parte son:

```
showNumberP1
updateBoardP1
copyMatrixP1
rotateMatrixLRP1
shiftNumbersLP1
addPairsLP1
```

En la segunda parte se tendrán que implementar las funcionalidades adicionales necesarias para completar todas las funcionalidades del juego del 2048.

Las subrutinas en ensamblador que tenéis que implementar o modificar para implementar el paso de parámetros en la Segunda Parte son:

```
showNumberP2
updateBoardP2
copyMatrixP2
rotateMatrixLRP2
shiftNumbersLP2
addPairsLP2
checkEndP2
```

Implementación de los movimientos:

Para implementar cada movimiento disponemos de 3 funcionalidades: rotar, desplazar y hacer parejas.

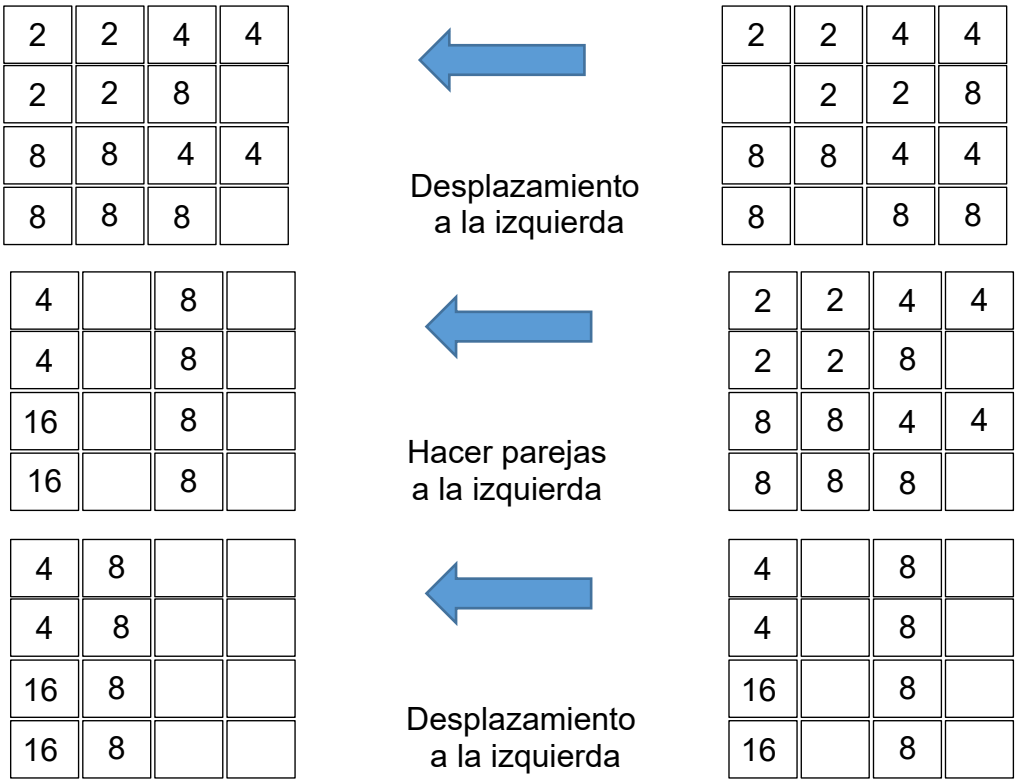
En la práctica implementaremos los 4 tipos de movimientos, izquierda, derecha, arriba y abajo, utilizando solo el desplazamiento a la izquierda (subrutina *shiftNumbersLP1*), hace parejas a la izquierda (subrutina *addPairsLP1*) y haciendo una o más rotaciones a la derecha o la izquierda de 90° de la matriz (subrutina *rotateMatrixLRP1* y copyMatrixP1), después de cada rotación se tiene que ejecutar la subrutina copyMatrixP1 porque la subrutina *rotateMatrixLRP1* deja los valores que se han rotado en la matriz *mRotated* y se tienen que volver a copiar a la matriz *m*.

Para cada tipo de movimiento haremos los pasos que se indican a continuación:

Movimiento a la izquierda:

- Desplazamiento a la izquierda.
- Hace parejas a la izquierda.
- Volver a hacer desplazamiento a la izquierda para juntar las parejas que se hayan hecho.

Ejemplo de un movimiento a la izquierda:



Para el resto de movimientos habrá que rotar la matriz para poder hacer los desplazamientos y las parejas hacia la izquierda y después dejarla como estaba.

Movimiento a la derecha:

- Rotación de 180° a la derecha = 2 rotaciones de 90° a la derecha (para cada rotación copiar nuevamente la matriz mRotated a m).
- Desplazamiento a la izquierda.
- Hacer parejas a la izquierda.
- Volver a hacer desplazamiento a la izquierda para juntar las parejas que se hayan hecho.
- Rotación de 180° a la izquierda = 2 rotaciones de 90° a la izquierda (para cada rotación copiar nuevamente la matriz mRotated a m).

Movimiento abajo:

- Rotación de 90° a la derecha = 1 rotación de 90° a la derecha (para cada rotación copiar nuevamente la matriz mRotated a m).
- Desplazamiento a la izquierda.
- Hacer parejas a la izquierda.
- Volver a hacer desplazamiento a la izquierda para juntar las parejas que se hayan hecho.
- Rotación de 90° a la izquierda = 1 rotación de 90° a la izquierda (para cada rotación copiar nuevamente la matriz mRotated a m).

Movimiento arriba:

- Rotación de 90° a la izquierda = 1 rotación de 90° a la izquierda (por cada rotación copiar nuevamente la matriz *mRotated* a m).
- Desplazamiento a la izquierda.
- Hacer parejas a la izquierda.
- Volver a hacer desplazamiento a la izquierda para juntar las parejas que se hayan hecho.
- Rotación de 90° a la derecha = 1 rotaciones de 90° a la derecha (por cada rotación copiar nuevamente la matriz mRotated a m).

En estos 3 casos hacemos rotaciones, y hay que volver a dejarla en la misma orientación que tenía.

Haremos la implementación de esta forma para evitar la implementación de cuatro subrutinas de desplazamiento y cuatro de hacer parejas

Veamos a continuación a modo de ejemplo qué operaciones se harían para hacer un desplazamiento abajo de los valores de la matriz.

Movimiento arriba

Tiene que hacer esto

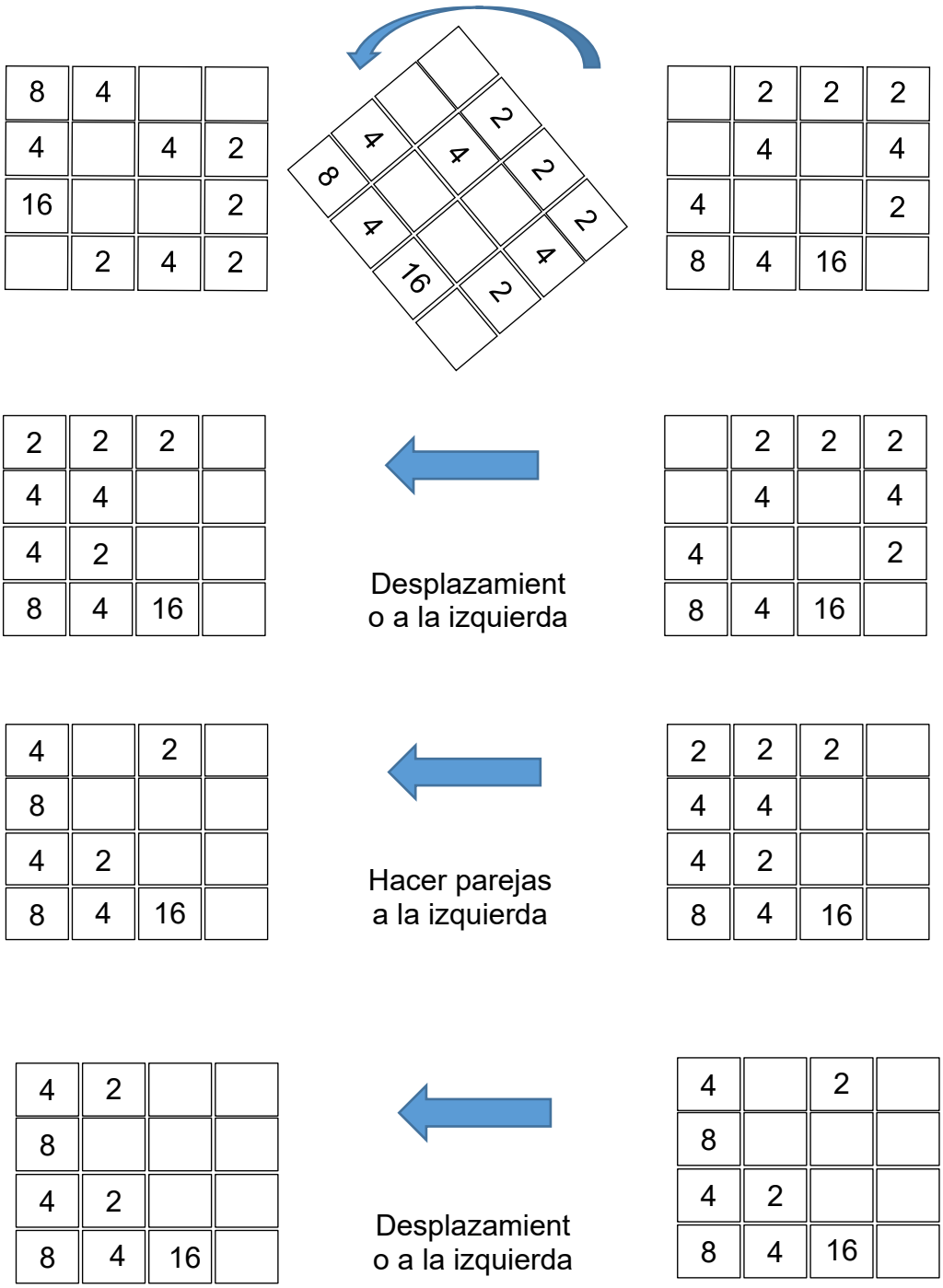
8	4		
4		4	2
16			2
	2	4	2

Moviment amunt

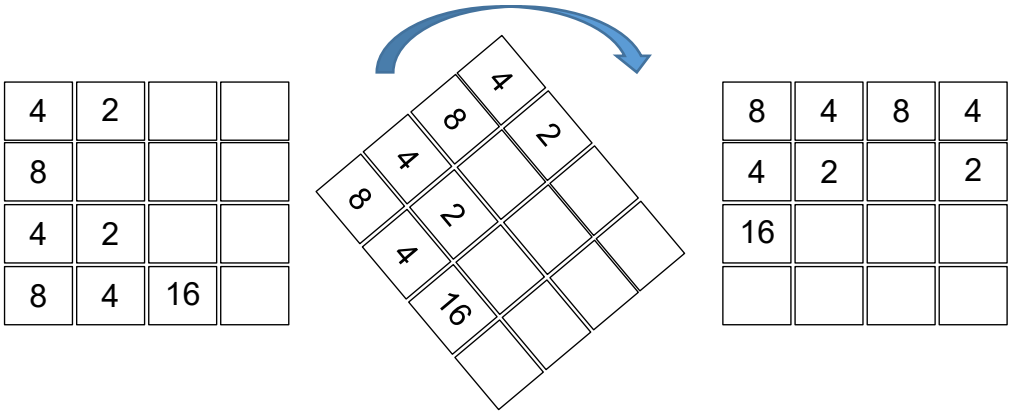
8	4	8	4
4	2		2
16			

Pasos que sigue el programa:

Rotación de 90° a la izquierda



Rotació de 90º a la dreta



Entrega de la práctica

La **primera parte** de la práctica (P1) tiene dos fechas de entrega. En la **primera entrega** se puede obtener una puntuación máxima (A). Si no se ha podido realizar la primera entrega o esta primera entrega no ha sido satisfactoria se podrá hacer una **segunda entrega**. En esta segunda entrega se puede obtener una calificación máxima de (B).

Si se ha superado la P1 en la primera entrega no es necesario volver a realizar la entrega de la P1 en la segunda entrega.

La **segunda parte** de la práctica (P2) tiene una única entrega, en la misma fecha de la segunda entrega de la P1.

Fecha límite Primera Entrega (P1):

Viernes, 10 de abril de 2026 a las 24:00:00

Data límite Segunda Entrega (P1 - P2):

Viernes, 15 de mayo de 2026 a las 24:00:00

La entrega debe realizarse a través de la actividad correspondiente del aula. Se debe entregar sólo un archivo con el código ensamblador bien comentado.

Evaluación de la práctica

La primera parte de la práctica (P1) tiene un peso del 70% de la nota de prácticas y la segunda parte de la práctica (P2) tiene un peso del 30% de la nota de prácticas.

Para evaluar la segunda parte de la práctica (P2) debe haberse superado la primera parte de la práctica (P1).

El esquema de entregas y calificación se puede resumir en la siguiente tabla.

Primera Entrega P1	**Superada A (10-9)**	**Superada A (9-10)**	**NO Superada NO Presentada**	**NO Superada NO Presentada**	**NO Superada NO Presentada**
Segunda Entrega P1	**No hay que entregar nada**	**No hay que entregar nada**	**Superada B (8-7,2)**	**Superada B (8-7,2)**	**NO Superada NO Presentada**
Entrega P2	**Superada A (10-9)**	**NO Superada NO Presentada**	**Superada A (10-9)**	**NO Superada NO Presentada**	**No se puede entregar**
Nota Final Prácticas	**A**	**B - C+**	**B - C+**	**C+**	**D - N**

Los alumnos que no superen la PRÁCTICA obtendrán un suspenso D (calificación: 0-2) o N si no se ha presentado en la nota final de prácticas y con esta nota no se puede aprobar la asignatura, por este motivo la PRÁCTICA es obligatoria.

Al principio del archivo de código ensamblador tenéis que indicar vuestro nombre y apellidos en la declaración de la variable *developer*.

La **PRÁCTICA** es una **actividad evaluable individual**, por lo tanto, no se pueden hacer comentarios muy detallados en el foro de la asignatura, se puede hacer una consulta sobre un error que tengáis al ensamblar el programa o algún detalle en concreto, pero no podéis poner el código de una subrutina o un bucle completo.

La práctica tiene que funcionar completamente para considerarse superada, **tiene que funcionar correctamente la opción del menú correspondiente al juego completo en ensamblador.**

Las otras opciones del menú son sólo para comprobar individualmente cada una de las subrutinas que hay que implementar. No es suficiente para aprobar la práctica que las opciones correspondientes a las subrutinas individuales funcionen.

Otro aspecto importante es la documentación del código que ayude a entenderlo mejor. No hay que explicar que hace cada instrucción (se da por entendido que quien lo lee sabe ensamblador) sino que hay que explicar a quina tarea de más alto nivel corresponde.